



大家好

欢迎加入 LightDir (光向) 研习社
欢迎大家一同探索开源共享的知识分享模式

今日内容



 作业·点评 答疑

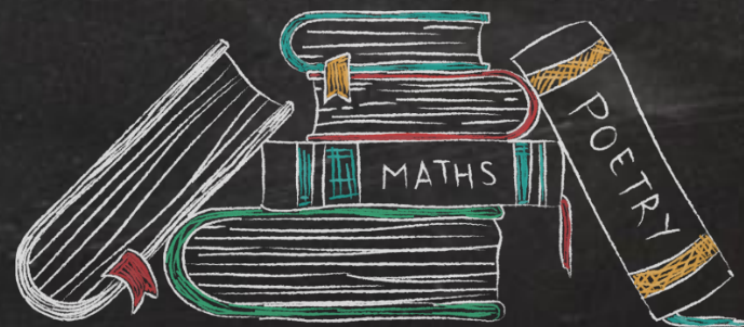
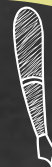
 结印·大乘·OldSchoolPro

 符文·大乘·OldSchoolPro

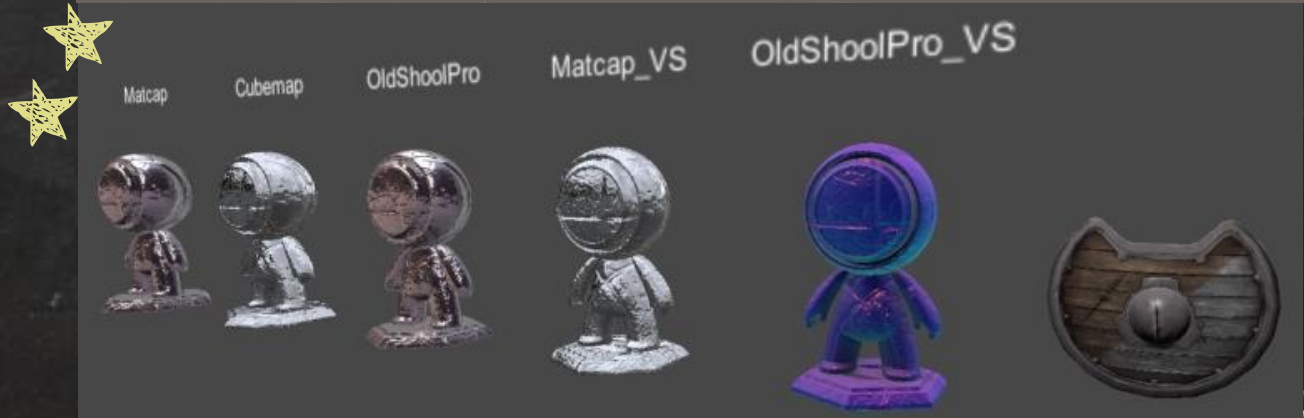
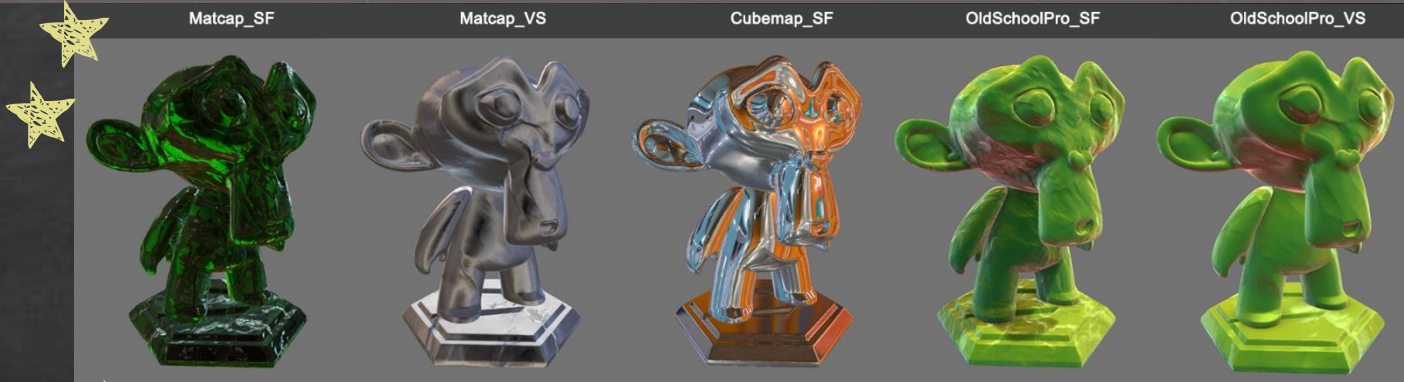
$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

1

作业·点评 答疑



01 一组



02 二组

Fresnel

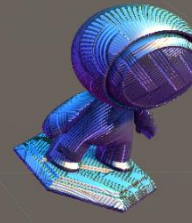
MatCap

MatCap_Code

Cubemap

OldSchoolPro

OldSchoolPro_Code



Matcap

CubeMap

OldSchoolP



normal_SF

normal_VC

matcap_SF

matcap_VC

cubemap_SF

OldSchoolPro_SF

OldSchoolPro_VC



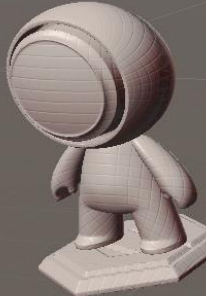
Matcap_SF

Cubemap_SF

OldSchoolPro_SF

Matcap

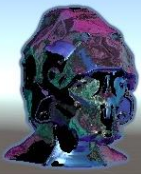
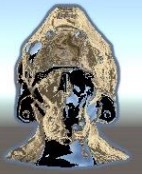
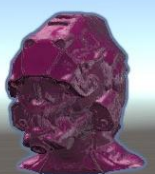
OldSchoolPro



109_matcap_sf

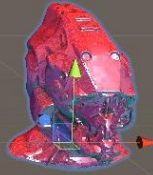
109_cubemap_sf

109_oldschoolpro_sf

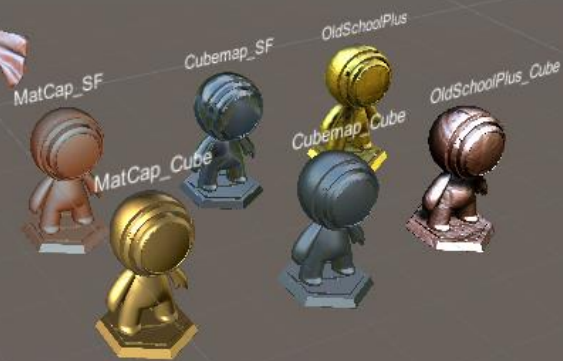
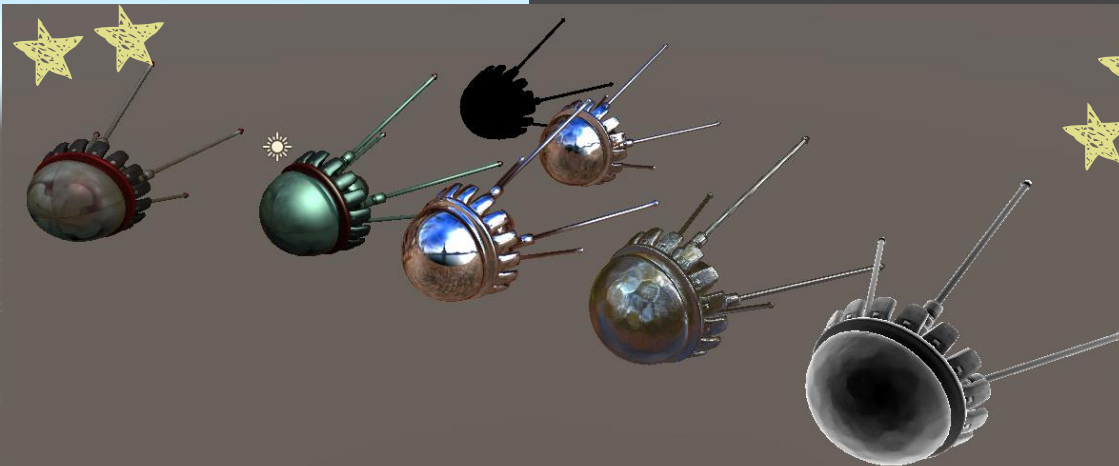
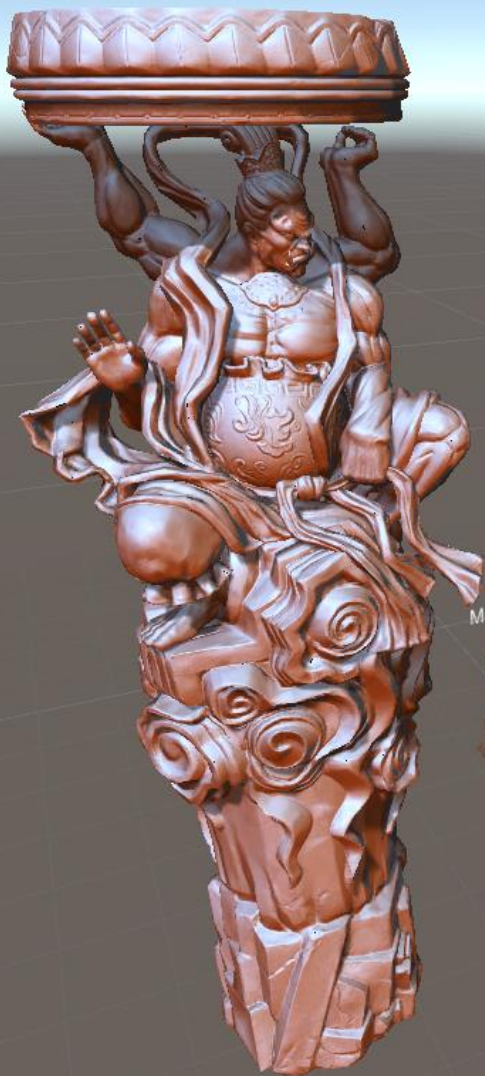


109_matcap_code

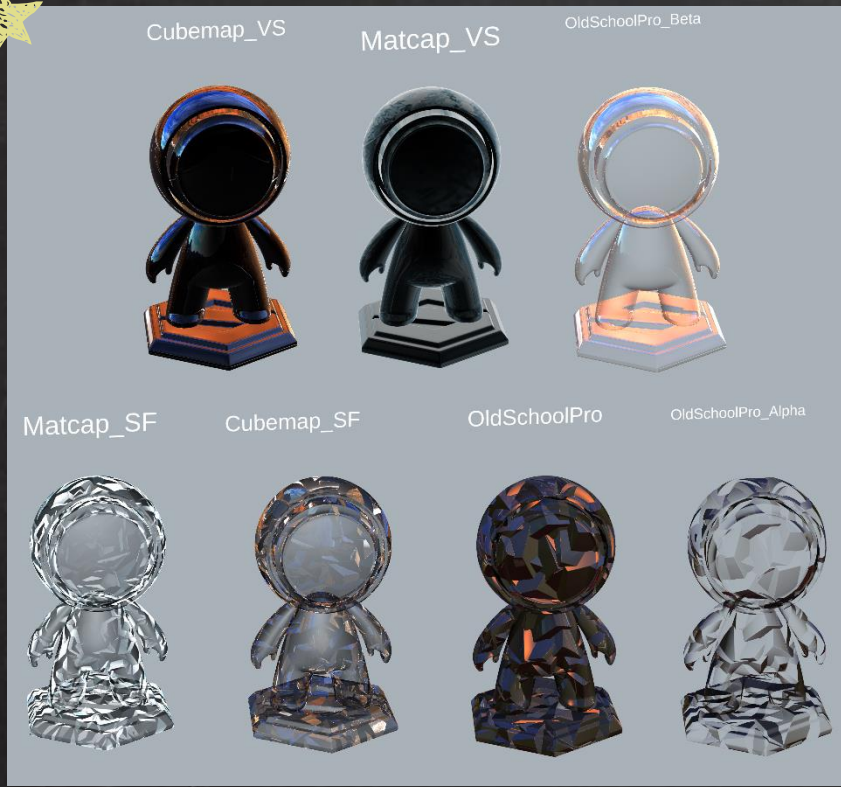
109_oldschoolpro_code



03 三组



04 四组



05 总结

姓名	代码作业	连连看作业	截图	打包	创意题
周川	✓	✓	✓	✓	
赵井才	✓	✓	✓	✓	
叶小芸	✓	✓	✓	✓	
赵翔	✓	✓	✓	✓	
张	✓	✓	✓	✓	模贴尝试 不讲
周翰林	✓	✓	✓	✓	
杨易	✓	✓	✓	✓	
廖宴楠	✓X	✓	✓	✓	
冯超越	✓	✓	✓	✓	模贴尝试 不讲
邵志刚	✓	✓	✓	✓	
申伏琳	✓	✓	✓	✓	模贴尝试 不讲
陈沛霖	✓	✓	✓	✓	模贴尝试 不讲
顾友海	✓	✓	✓	✓	模贴尝试 不讲
冷翰林	✓	✓	✓	✓	模贴尝试 可讲
王岩	✓	✓	✓	✓	
宋歌	✓	✓	✓	✓	
冯晓晨	✓	✓	✓	✓	
罗陈	✓	✓	✓	✓	
昊					
沈术磐	✓	✓	✓	✓	

作业情况：

- 本周作业有点多，同学提交的创意尝试较少；
- 部分同学代码还是比较吃力，需要时间巩固消化；

周日计划：

- 不讲新内容，归纳复习1~9所学内容；
- 布置一个大作业，Shader贴图效果联调的工作状态一定要体会到；

06 答疑

- 啥时候能讲金属度和粗糙度贴图的使用？加上这两是不是就是魔改PBR？
 - PBR的由来之一就是对表面参数的概括；可以看到Shader的功能加的越多，面板参数也堆的越多；怎么通过寻找参数之间的物理关系，来减少面板参数，是PBR的内容之一；所以不能脱离物理模型谈金属度，粗糙度贴图；
 - 以PBR基于物理为核修改，叫魔改PBR；把一些PBR用到的BRDF或者纹理名称移植到传统模型，那还是传统模型，不叫魔改PBR；
 - 手游PBS中什么叫基于物理为核，1) 基于物理的光能守恒；2) 基于物理的表面属性归纳；3) 微表面理论；
 - 现在谈还太早；
- 为什么高低模不变，旋转UV，烘焙的法线贴图会变？
 - 法线贴图是记录在切线空间的法线信息；
 - 切线空间的主副切线方向可直观理解为，贴图在表面处的uv轴方向；
 - 模型不变，切线空间变了，记录的法线信息也会改变；
- 能否用所学课程创建无缝Shader？
 - 预计算的东西（Lightmap 曲率图 等等）就是拿内存换性能；这个交换一般来说都是值得的；
 - 所以，不要用Shader去计算，资源上就可以解决的事情；

07 答疑

- mul(),dot(),*各种乘法的区别? float2,3,4怎么用? 代码如何实现贴图的TilingOffset?
 - 周日复习课一并拿出来讲;
- SF可以嵌套其他Shader吗? SF相关的快捷键操作?
 - 职业生涯SF就只会了一周(然后就转代码了), 真不太熟, 抱歉了;
 - SF是学习代码的辅助, 我们熟悉代码之后是要扔掉它的, 不用太过纠结;
- 可以整理个能力模型吗? 从入门>进阶>资深这种:
 - 一般都是领导和HR才来问我这种问题, 也有现成的答案来帮助他们按图索骥, 但是作为从业人员建议关心下入门标准就好;
 - 能力模型每个公司, 甚至每个工作室的标准都不一样; 且随时间激烈改变, 看太长远不如关注当下; 最简单的, 找你想去公司对应岗位的招聘简介;
 - 不要迷信仙人指路, 小心到你这成了刻舟求剑; 做好你实际面对的每一个选择, 跨过你实际面对的每一道坎, 千万不要在自己或者别人假想中的职业生涯里, 无效的投入时间;
 - 如上, 没有面对选择和挑战的环境, 那就创造环境;

08 作业回顾

見 209 课件.

- 连连看作业:

- Matcap, Cubemap;
- OldSchoolPro = OldSchoolPlus + Normal + Cubemap;

- 符文作业:

- Matcap;
- OldSchoolPro;

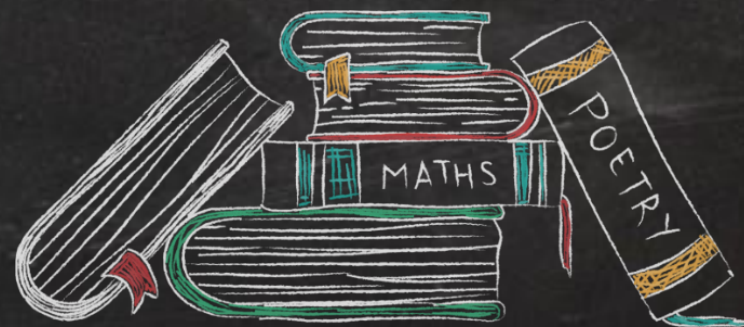
- 创意作业:

- 改进你的完备光照模型;

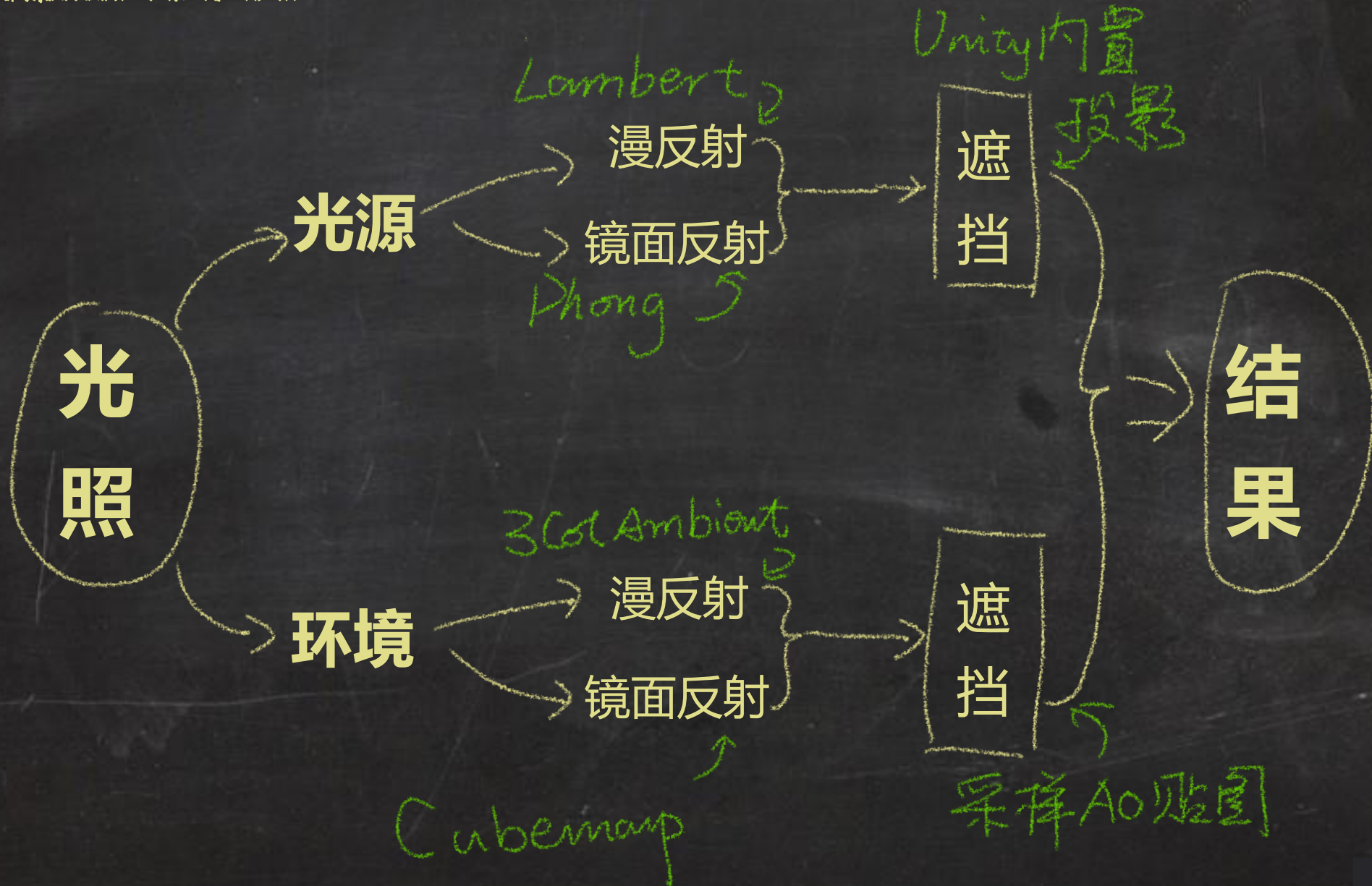
演示

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

2 结印·OldSchoolPro

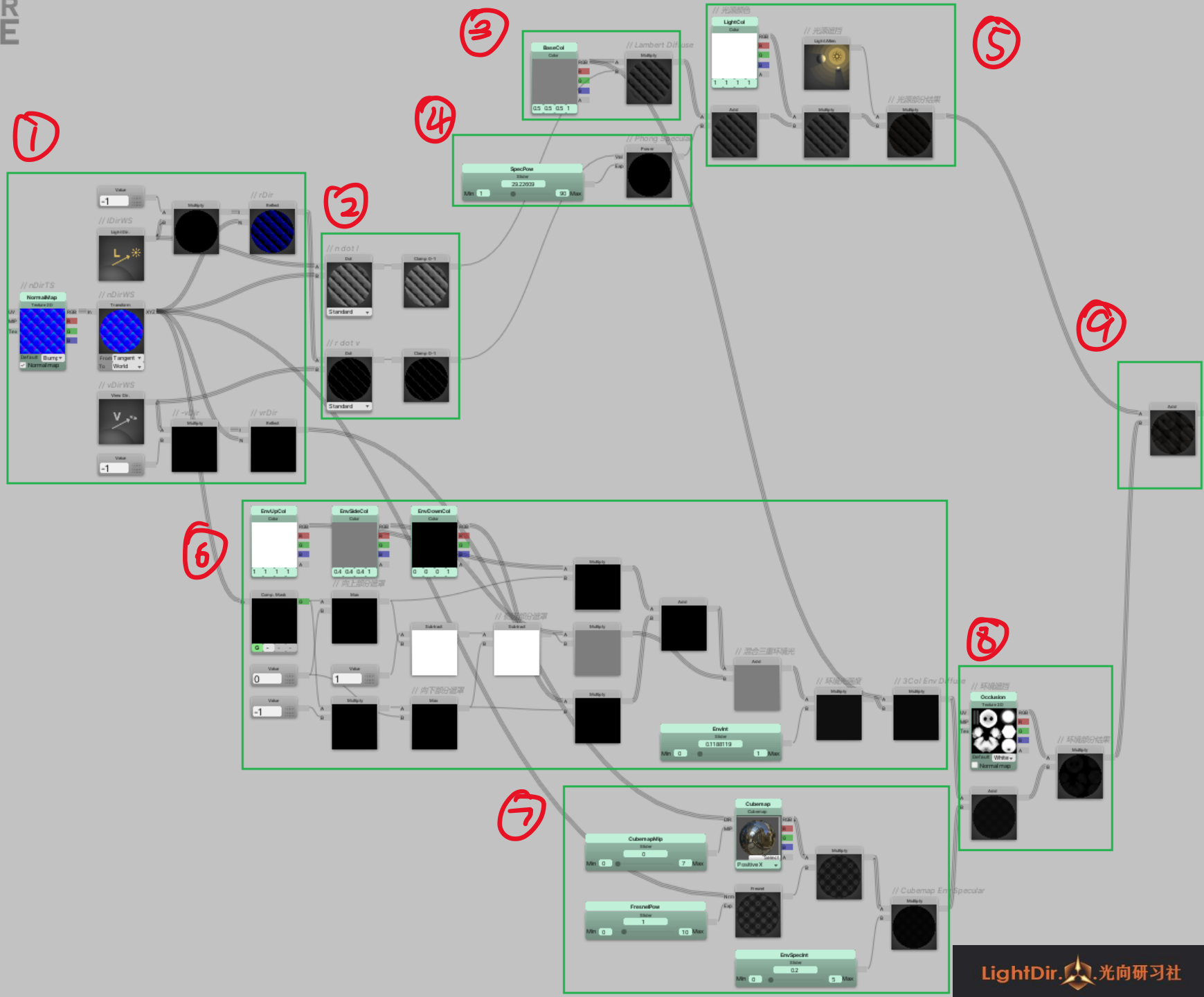


01 OldSchoolPro构成

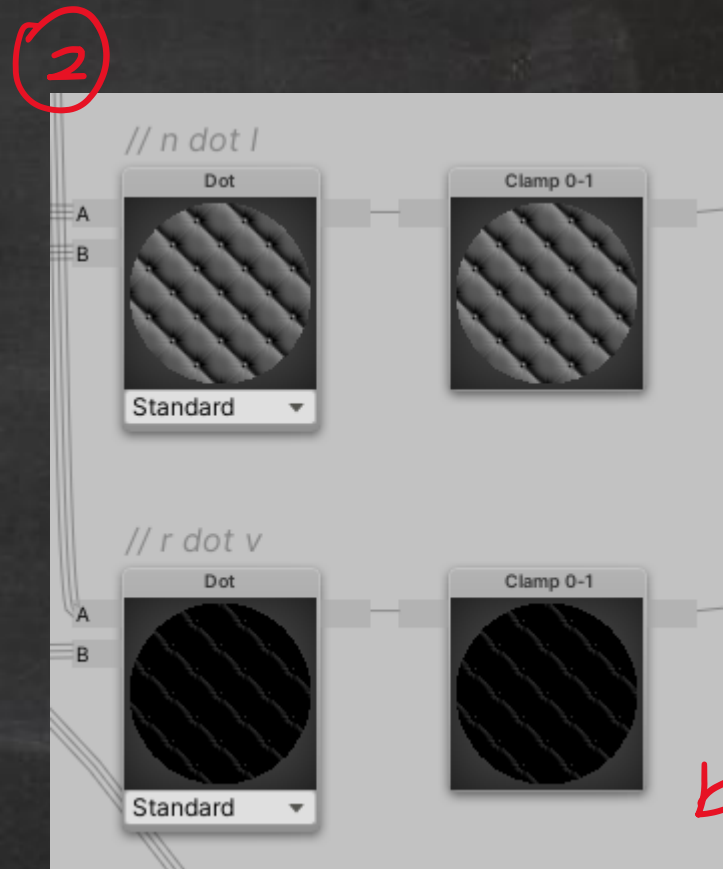
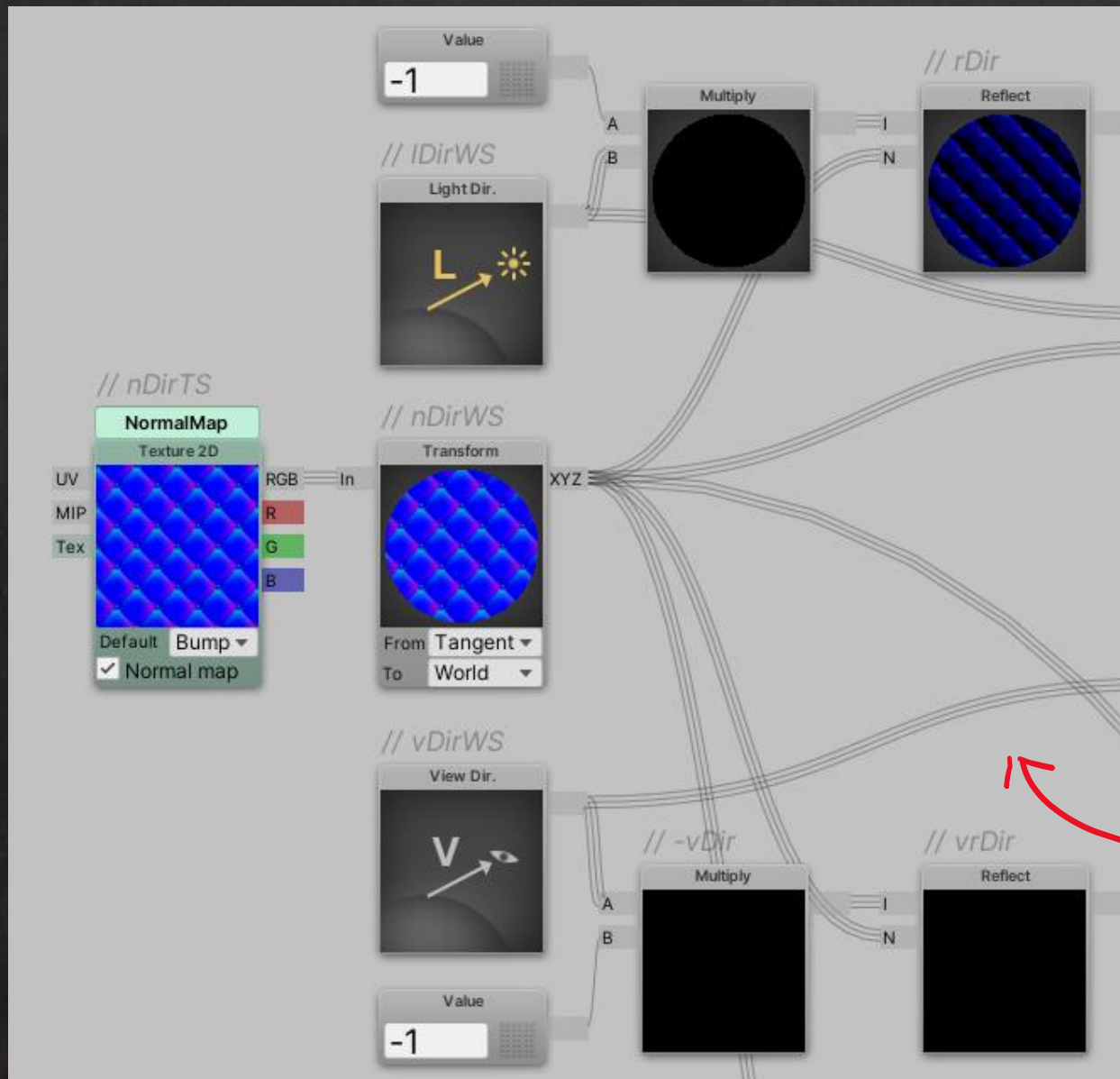


02 结印·OldSchoolPro

1. 向量准备: nDir, lDir, vDir, rDir, vrDir;
2. 中间量准备: ndotl, rdotv;
3. Lambert 光源漫反射;
4. Phong 光源镜面反射;
5. 光源反射混合;
6. 3Col 环境漫反射;
7. Cubemap 环境镜面反射;
8. 环境反射混合;
9. 光源, 环境反射叠加;

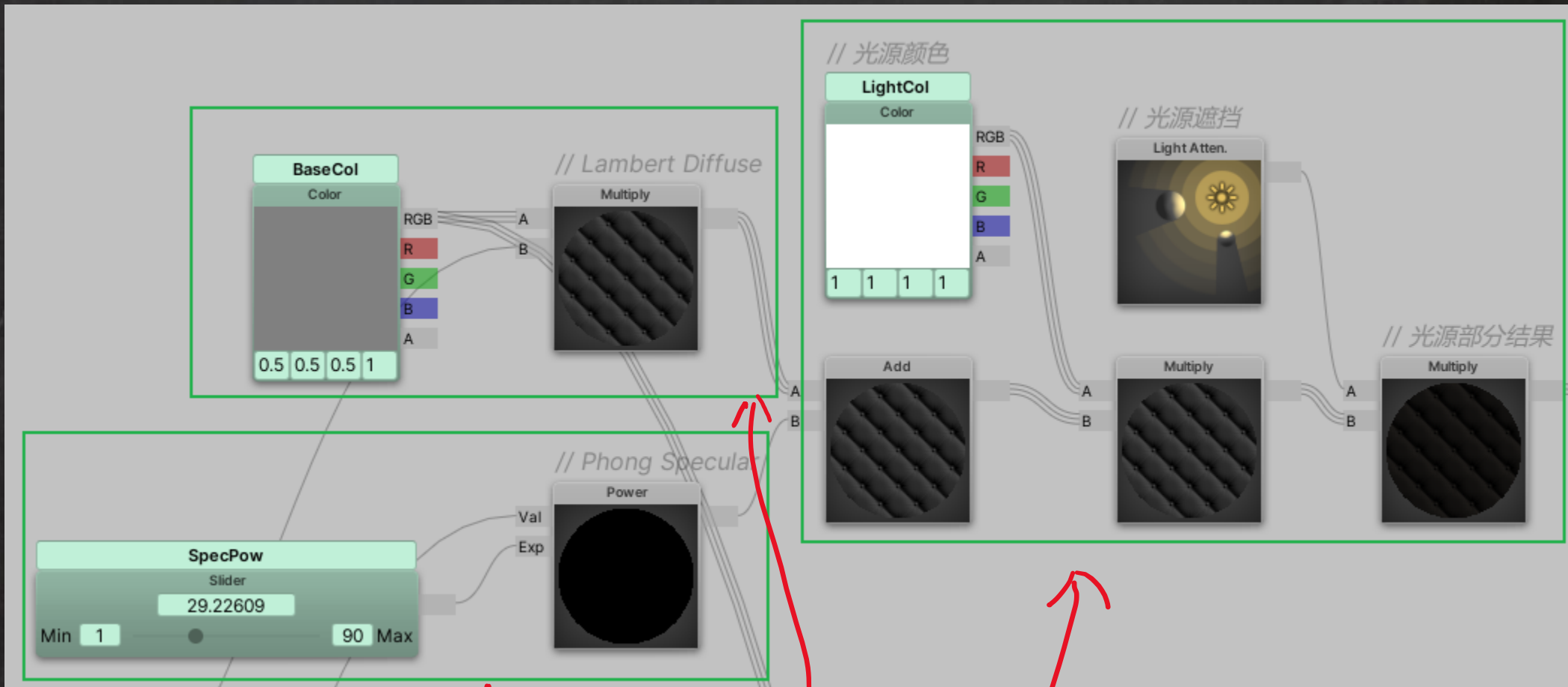


03 结印·OldSchoolPro



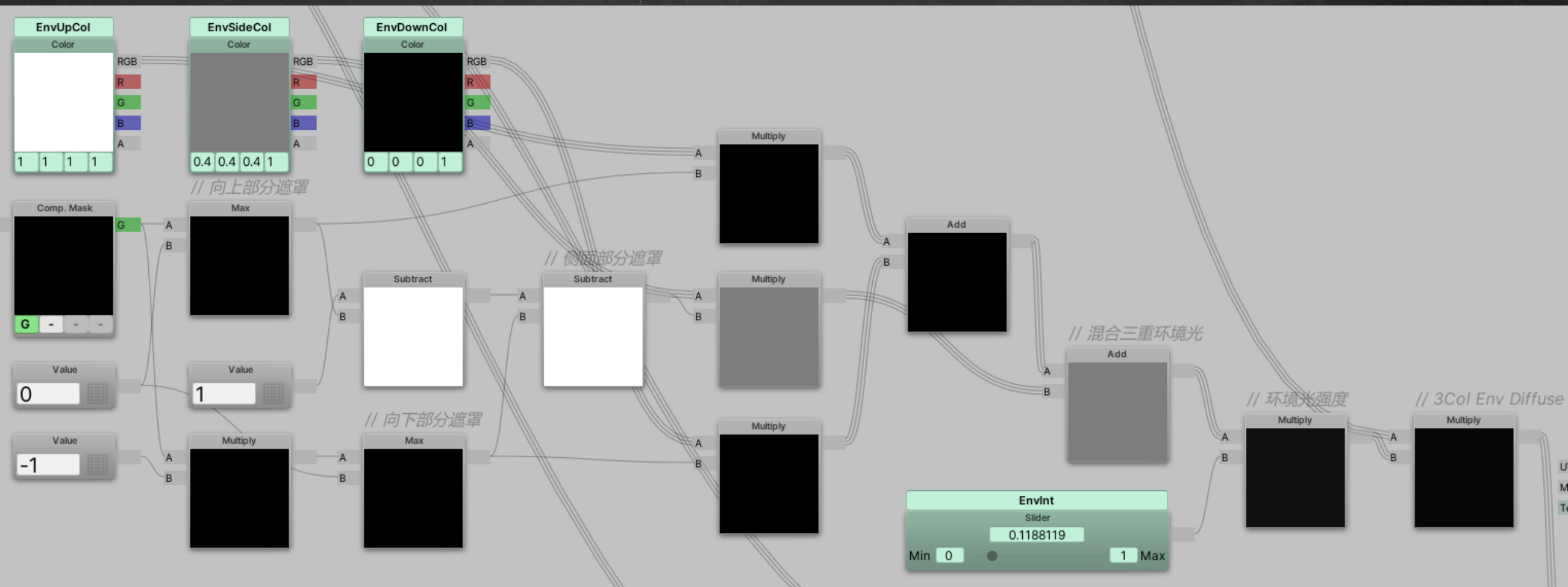
- 向量准备: nDir, lDir, vDir, rDir, vrDir;
- 中间量准备: ndotl, rdotv;

04 结印·OldSchoolPro



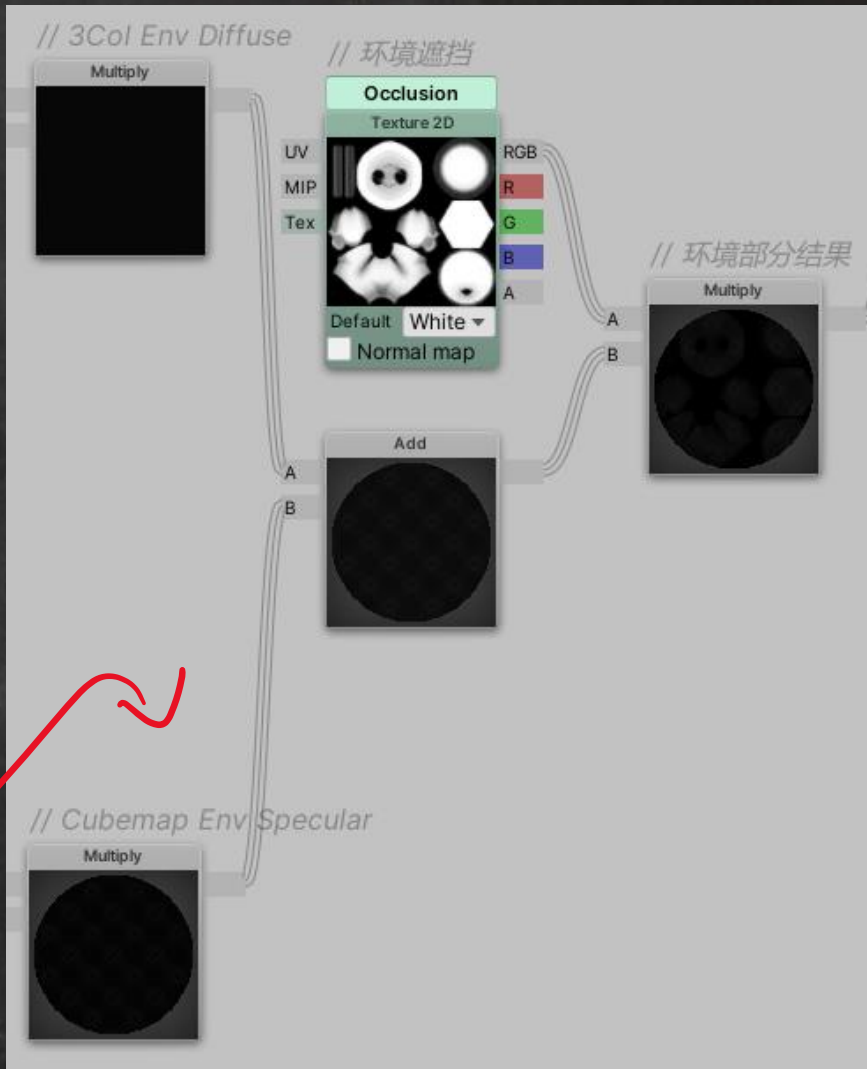
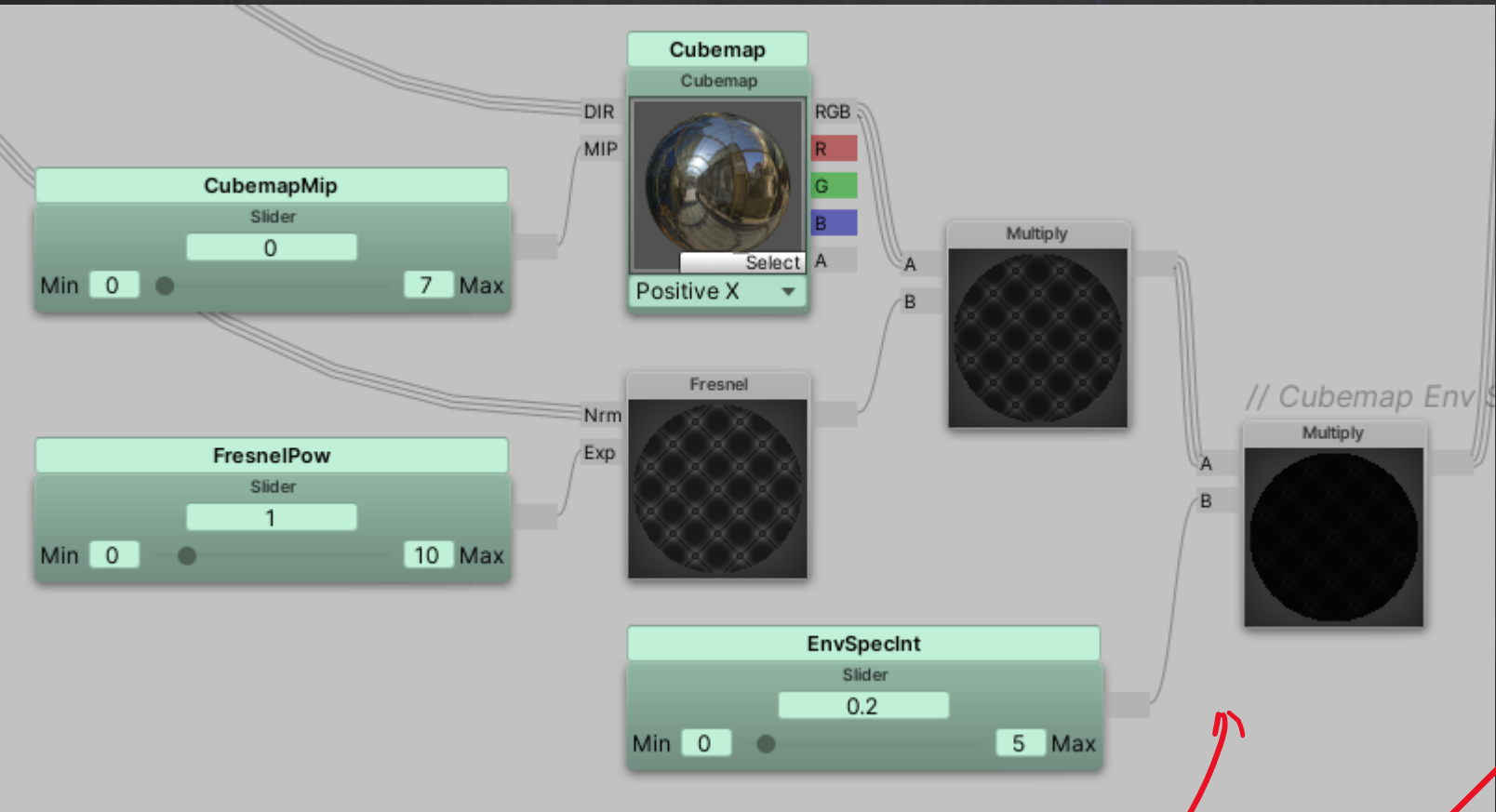
- Lambert 光源漫反射;
- Phong 光源镜面反射;
- 光源反射混合;

05 结印·OldSchoolPro



- 3Col 环境漫反射;

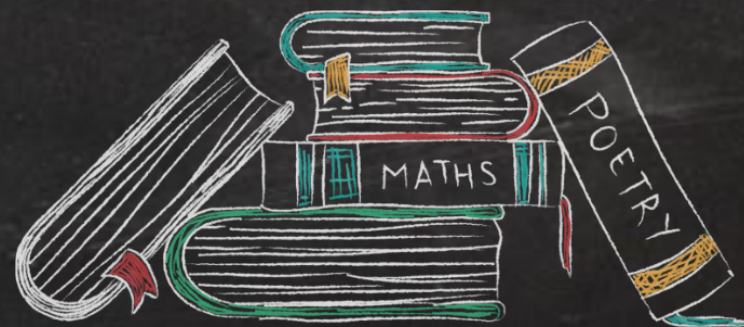
06 结印·OldSchoolPro



- Cubemap 环境镜面反射;
- 环境反射混合;

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

3 符文·OldSchoolPro



01 符文·OldSchoolPro

1. 为巩固所学内容, 从FlatCol作为模板开始;
2. 复制粘贴, 修改Shader路径名;
3. 定义面板参数
 - [Header(XXX)] 排版

Shader "AP01/L09/OldSchoolPro" {
 Properties {
 [Header(Texture)]
 _MainTex ("RGB:基础颜色 A:环境遮罩", 2D) = "white" {}
 _NormTex ("RGB:法线贴图", 2D) = "bump" {}
 _SpecTex ("RGB:高光颜色 A:高光次幂", 2D) = "gray" {}
 _EmitTex ("RGB:环境贴图", 2d) = "black" {}
 _Cubemap ("RGB:环境贴图", cube) = "_Skybox" {}
 [Header(Diffuse)]
 _MainCol ("基本色", Color) = (0.5, 0.5, 0.5, 1.0)
 _EnvDiffInt ("环境漫反射强度", Range(0, 1)) = 0.2
 _EnvUpCol ("环境天顶颜色", Color) = (1.0, 1.0, 1.0, 1.0)
 _EnvSideCol ("环境水平颜色", Color) = (0.5, 0.5, 0.5, 1.0)
 _EnvDownCol ("环境地表颜色", Color) = (0.0, 0.0, 0.0, 0.0)
 [Header(Specular)]
 _SpecPow ("高光次幂", Range(1, 90)) = 30
 _EnvSpecInt ("环境镜面反射强度", Range(0, 5)) = 0.2
 _FresnelPow ("菲涅尔次幂", Range(0, 5)) = 1
 _CubemapMip ("环境球Mip", Range(0, 7)) = 0
 [Header(Emission)]
 _EmitInt ("自发光强度", range(1, 10)) = 1
 }
}

// 追加投影相关包含文件
#include "AutoLight.cginc"
#include "Lighting.cginc"
#pragma multi_compile_fwdbase_fullshadows
#pragma target 3.0
// 输入参数
// Texture
uniform sampler2D _MainTex;
uniform sampler2D _NormTex;
uniform sampler2D _SpecTex;
uniform sampler2D _EmitTex;
uniform samplerCUBE _Cubemap;
// Diffuse
uniform float3 _MainCol;
uniform float _EnvDiffInt;
uniform float3 _EnvUpCol;
uniform float3 _EnvSideCol;
uniform float3 _EnvDownCol;
// Specular
uniform float _SpecPow;
uniform float _FresnelPow;
uniform float _EnvSpecInt;
uniform float _CubemapMip;
// Emission
uniform float _EmitInt;

4. 为产生投影, 包含以下文件:

- AutoLight.cginc
- Lighting.cginc

5. 与材质面板——对应声明输入参数;

02 符文·OldSchoolPro

6. 输入结构包含：

- POSITION 顶点位置；
- TEXCOORD0 纹理UV0；
- NORMAL 法线方向；
- TANGENT 切线方向；

7. 输出结构包含：

- pos 屏幕顶点位置；
- uv0 纹理UV0；
- posWS 世界空间位置
- nDirWS 世界空间法线
- tDirWS 世界空间切线
- bDirWS 世界空间副切线
- LIGHTING_COORD() 投影信息

8. 顶点Shader，编写对应输入输出结构内容；

```
// 输入结构
struct VertexInput {
    ⑥ { float4 vertex : POSITION; // 顶点信息 Get ✓
        float2 uv0 : TEXCOORD0; // UV信息 Get ✓
        float4 normal : NORMAL; // 法线信息 Get ✓
        float4 tangent : TANGENT; // 切线信息 Get ✓
    };
// 输出结构
struct VertexOutput {
    ⑦ { float4 pos : SV_POSITION; // 屏幕顶点位置
        float2 uv0 : TEXCOORD0; // UV0
        float4 posWS : TEXCOORD1; // 世界空间顶点位置
        float3 nDirWS : TEXCOORD2; // 世界空间法线方向
        float3 tDirWS : TEXCOORD3; // 世界空间切线方向
        float3 bDirWS : TEXCOORD4; // 世界空间副切线方向
        LIGHTING_COORDS(5,6) // 投影相关
    };
// 输入结构>>>顶点Shader>>>输出结构
VertexOutput vert (VertexInput v) {
    VertexOutput o = (VertexOutput)0;
    ⑧ { o.pos = UnityObjectToClipPos( v.vertex ); // 新建输出结构
        o.uv0 = v.uv0; // 顶点位置 OS>CS
        o.posWS = mul(unity_ObjectToWorld, v.vertex); // 传递UV
        o.nDirWS = UnityObjectToWorldNormal(v.normal); // 顶点位置 OS>WS
        o.tDirWS = normalize(mul(unity_ObjectToWorld, float4(v.tangent.xyz, 0.0)).xyz); // 法线方向 OS>WS
        o.bDirWS = normalize(cross(o.nDirWS, o.tDirWS) * v.tangent.w); // 切线方向 OS>WS
        o.bDirWS = normalize(cross(o.nDirWS, o.tDirWS) * v.tangent.w); // 副切线方向
        TRANSFER_VERTEX_TO_FRAGMENT(o) // 投影相关
    }
    return o; // 返回输出结构
}
```

02 符文·OldSchoolPro

9. 像素Shader, 五段式写法:

1. 向量准备
2. 中间量准备
3. 纹理采样
4. 光照模型
5. 返回值

```
// 输出结构>>>像素
float4 frag(VertexOutput i) : COLOR {
    // 准备向量
    float3 nDirTS = UnpackNormal(tex2D(_NormTex, i.uv0)).rgb;
    float3x3 TBN = float3x3(i.tDirWS, i.bDirWS, i.nDirWS);
    float3 nDirWS = normalize(mul(nDirTS, TBN));
    float3 vDirWS = normalize(_WorldSpaceCameraPos.xyz - i.posWS.xyz);
    float3 vrDirWS = reflect(-vDirWS, nDirWS);
    float3 lDirWS = _WorldSpaceLightPos0.xyz;
    float3 lrDirWS = reflect(-lDirWS, nDirWS);
    // 准备点积结果
    float ndotl = dot(nDirWS, lDirWS);
    float vdotr = dot(vDirWS, lrDirWS);
    float vdotn = dot(vDirWS, nDirWS);
    // 采样纹理
    float4 var_MainTex = tex2D(_MainTex, i.uv0);
    float4 var_SpecTex = tex2D(_SpecTex, i.uv0);
    float3 var_EmitTex = tex2D(_EmitTex, i.uv0).rgb;
    float3 var_Cubemap = texCUBElod(_Cubemap, float4(vrDirWS, lerp(_CubemapMip, 0.0, var_SpecTex.a))).rgb;
    // 光照模型(直接光照部分)
    float3 baseCol = var_MainTex.rgb * _MainCol;
    float lambert = max(0.0, ndotl);
    float specCol = var_SpecTex.rgb;
    float specPow = lerp(1, _SpecPow, var_SpecTex.a);
    float phong = pow(max(0.0, vdotr), specPow);
    float shadow = LIGHT_ATTENUATION(i);
    float3 dirLighting = (baseCol * lambert + specCol * phong) * _LightColor0 * shadow;
    // 光照模型(环境光照部分)
    float upMask = max(0.0, nDirWS.g); // 获取朝上部分遮罩
    float downMask = max(0.0, -nDirWS.g); // 获取朝下部分遮罩
    float sideMask = 1.0 - upMask - downMask; // 获取侧面部分遮罩
    float3 envCol = _EnvUpCol * upMask +
        _EnvSideCol * sideMask +
        _EnvDownCol * downMask; // 混合环境色
    float fresnel = pow(max(0.0, 1.0 - vdotn), _FresnelPow); // 菲涅尔
    float occlusion = var_MainTex.a;
    float3 envLighting = (baseCol * envCol * _EnvDiffInt + var_Cubemap * fresnel * _EnvSpecInt * var_SpecTex.a) * occlusion;
    // 光照模型(自发光部分)
    float3 emission = var_EmitTex * _EmitInt;
    // 返回结果
    float3 finalRGB = dirLighting + envLighting + emission;
    return float4(finalRGB, 1.0);
}
```




Thanks